

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ АНАЛОГОВ	9
1.1 Обзор исследований и моделей машинного обучения для решения аналогичных задач	9
1.1.1 Применение классических и глубоких автокодировщиков для оценки состояния оборудования	9
1.1.2 Вариационные автокодировщики в задачах вероятностного прогнозирования и генерации временных рядов	9
1.1.3 Сравнительный анализ генеративных архитектур (GAN, Diffusion, VAE) для моделирования циклических процессов	10
1.1.4 Адаптация функции ELBO и амортизированный вывод для многошагового прогнозирования состояний	11
1.2 Критерии оценки и валидации генеративных моделей в промышленной диагностике	11
1.3 Научная новизна	12
1.4 Теоретическая и практическая значимость	12
1.5 Положения, выносимые на защиту	14
1.6 Выводы	14
2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ МОДЕЛИРОВАНИЯ ДАННЫХ .	16
2.1 Постановка задачи	16
2.1.1 Описание входных данных	17
2.1.2 Описание выходных данных	19
3 ПРОВЕДЕНИЕ ЭКСПЕРИМЕНТОВ И АНАЛИЗ РЕЗУЛЬТАТОВ . .	21
3.1 Описание исследуемого набора данных и предварительная обработка	21
3.2 Архитектура экспериментальных моделей и программная реализация	21
3.3 Методология и протокол обучения моделей	21
3.4 Система метрик и методы стресс-тестирования	21
3.5 Анализ и интерпретация экспериментальных результатов	21
3.6 Выводы	21
4 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ МОДЕЛИРОВАНИЯ	21
4.1 Архитектура программного комплекса	21

3.2 Структура Python-библиотеки и организация пакетов	23
3.3 Описание функциональных модулей	26
3.3.1 Модуль чтения и обработки данных	26
3.3.2 Модуль предобработки и пайплайна	28
3.3.2 Модуль предобработки и пайплайна	28
3.3.3 Модуль нейросетевых моделей	30
3.3.4 Модуль оценки метрик	32
3.3.5 Модуль управления экспериментами	34
3.5 Направления и возможности функционирования системы	36
3.5.1 Пайплайн компонентов	37
3.5.2 Пайплайн последовательности обучения	38
3.5.2 Пайплайн последовательности обучения	38
3.5.3 Пайплайн последовательности инференса	40
3.6 Интеграция с внешними сервисами и требования к развертыванию .	40
3.6 Интеграция с внешними сервисами и требования к развертыванию .	40
3.6.1 Интеграция с платформой MLflow	40
3.6.2 Контейнеризация и развертывание	41
3.6.3 Аппаратные и программные требования	41
3.6.4 Программный интерфейс взаимодействия	42
3.7 Выводы	43

ВВЕДЕНИЕ

В современных условиях эксплуатации сложных технологических систем крайне важно не только учитывать текущее состояние составляющих систему модулей, но и уметь прогнозировать их поведение на несколько эксплуатационных циклов вперёд. Прогнозирование выхода параметров за допустимые границы позволяет не только снизить риски аварийной эксплуатации, но и повысить общую эффективность производственного процесса, предотвратив экономические издержки, связанные с незапланированными простоями или преждевременным снятием с эксплуатации ещё ресурсных узлов.

Оценка и моделирование текущего состояния оборудования являются ключевыми факторами перехода к предиктивному обслуживанию. Своевременное формирование вероятностных сценариев развития состояния дает возможность использовать оборудование с максимальной эффективностью и оптимизировать графики технического обслуживания. В работах [2–10] рассматривается решение проблем обнаружения аномалий и прогнозирования остаточного ресурса технологического оборудования с использованием моделей глубокого обучения. Особое внимание в современных исследованиях уделяется генеративным архитектурам, способным моделировать распределения временных рядов и синтезировать правдоподобные траектории развития отказов.

Особенностью данной работы является применение вариационного автокодировщика (VAE) в качестве основы системы. Модель решает две сопряженные задачи: классическую задачу восстановления текущих параметров и задачу генерации последующих n эксплуатационных циклов. Амортизация параметров латентного распределения z выполняется не по полному временному ряду, а на основе начальных n циклов, что позволяет эффективно оперировать скалярными показателями состояния St (не являющимися векторами) и генерировать элементы выходной последовательности. Математический аппарат алгоритма базируется на адаптации вариационной нижней оценки (ELBO) под задачу последовательного прогнозирования.

Данная работа имеет практическую значимость в областях производства и эксплуатации технологически сложного оборудования, в частности турбовентиляторных двигателей и промышленных агрегатов. Её применение

позволяет формировать прогнозы будущих показателей циклов, выявлять скрытые закономерности деградации и поддерживать принятие решений в условиях неполных или зашумленных данных.

В связи с этим целью данной работы являлось снижение эксплуатационных рисков и повышение экономической эффективности за счет создания системы моделирования состояния оборудования на базе вариационного автокодировщика, обеспечивающей генерацию последующих циклов и точную оценку технического состояния.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ АНАЛОГОВ

1.1 Обзор исследований и моделей машинного обучения для решения аналогичных задач

1.1.1 Применение классических и глубоких автокодировщиков для оценки состояния оборудования

В работе Jia Guo, Guannan Liu, Yuan Zuo, Junjie Wu под названием “An Anomaly Detection Framework Based on Autoencoder and Nearest Neighbor” [2] исследователи предлагают архитектуру АЕКNN, объединяющую автокодировщик и метод k-ближайших соседей. На этапе обучения модель тренируется исключительно на данных штатного режима. Сжатое представление в латентном слое используется как вход для k-NN классификатора. При тестировании аномалии выявляются по отклонению расстояния до ближайших соседей в скрытом пространстве. Эффективность подхода подтверждается на промышленных датасетах, однако метод не предусматривает генерацию будущих состояний и работает в режиме классификации «штатный/аварийный» без оценки вероятностной неопределенности.

В статье «Обнаружение аномальных событий на хосте с использованием автокодировщика» авторы Гурина А. О., Гузев О. Ю., Елисеев В. Л. предлагают метод построения модели нормального поведения системы на основе двух параллельных автокодировщиков с различной архитектурой. Критерием аномальности выступает превышение порога мгновенной ошибки реконструкции (IRE), рассчитываемого отдельно для каждого декодера. Метод демонстрирует высокую чувствительность к редким событиям, но опирается на детерминированную реконструкцию, что ограничивает его применимость для стохастических процессов износа технологического оборудования.

1.1.2 Вариационные автокодировщики в задачах вероятностного прогнозирования и генерации временных рядов

В работе Kingma and Welling “Auto-Encoding Variational Bayes” [16] заложены теоретические основы VAE, где максимизация вариационной ниж-

ней оценки (ELBO) обеспечивает одновременное обучение кодировщика, декодировщика и регуляризацию латентного пространства. Применительно к телеметрии оборудования данная архитектура позволяет оценивать не только точечное восстановление параметров, но и дисперсию реконструкции, что напрямую связано с надежностью прогноза.

Исследование Chung et al. “Deep Generative Models for Time Series Forecasting” демонстрирует применение рекуррентного VAE (VRNN) для многошагового прогнозирования. Авторы показывают, что аморизованный вывод параметров распределения z по скользящему окну данных позволяет генерировать правдоподобные траектории развития параметров. Однако в работе используется векторное представление состояния на каждом шаге, что увеличивает вычислительную нагрузку и усложняет интерпретацию скалярных индикаторов работоспособности

1.1.3 Сравнительный анализ генеративных архитектур (GAN, Diffusion, VAE) для моделирования циклических процессов

В обзоре Goodfellow et al. по генеративным состязательным сетям и последующих работах по Diffusion-моделям отмечается высокое качество генерации сложных распределений. Однако для задач прогнозирования циклов технологического оборудования эти архитектуры демонстрируют существенные ограничения: GAN страдают от нестабильности обучения (mode collapse) и отсутствия явной вероятностной интерпретации латентных переменных, что затрудняет калибровку порогов аварийных состояний. Diffusion-модели требуют многоэтапного обратного процесса генерации, что критично увеличивает время инференса и делает их неоптимальными для систем мониторинга в реальном времени. В работе Sohn et al. “Learning Structured Output Representation using Deep Conditional Generative Models” [20] показано, что условные VAE (CVAE) обеспечивают стабильную оптимизацию и явную связь между входным контекстом (текущие n циклов) и выходной последовательностью. Это позволяет формировать прогнозы с оценкой доверительных интервалов, что напрямую соответствует требованиям к системам предиктивного обслуживания.

1.1.4 Адаптация функции ELBO и амортизированный вывод для многошагового прогнозирования состояний

В исследованиях по байесовским методам прогнозирования [21, 22] отмечается, что стандартная формулировка ELBO требует модификации для последовательных данных. Авторы предлагают амортизировать параметры распределения не по полному временному ряду, а по начальному окну из циклов, что снижает риск переобучения и позволяет декодировщику генерировать последующие элементы на основе фиксированного латентного представления. В работе Zheng et al. “Variational Autoencoder for Remaining Useful Life Prediction” скалярный показатель остаточного ресурса выводится как функция от дисперсии реконструкции и расстояния в латентном пространстве. Авторы подчеркивают, что не является вектором признаков, а представляет собой агрегированную метрику, вычисляемую на основе вероятностного вывода. Такой подход позволяет отделить задачу генерации телеметрии от задачи оценки состояния, сохраняя при этом их математическую связность через общий ELBO.

1.2 Критерии оценки и валидации генеративных моделей в промышленной диагностике

Валидация генеративных моделей для временных рядов оборудования требует сочетания поточечных и структурных метрик. В работах [24, 25] базовой метрикой выступает поточечная MSE (Mean Squared Error), вычисляемая для каждого шага прогнозирования: размерность вектора наблюдений. Для учёта фазовых сдвигов и нелинейных деформаций временных осей дополнительно применяются метрики DTW (Dynamic Time Warping) и MAE. В исследовании показано, что для оценки качества генерации будущих циклов недостаточно лишь минимизации MSE, так как модель может «усреднять» распределение. Поэтому вводится критерий коэффициента детерминации по каждому прогнозируемому циклу и F1-score классификации режимов, где порог аномальности определяется на основе распределения ошибок реконструкции и расстояний Махаланобиса в латентном пространстве. Комплексное применение данных метрик позволяет объективно оценить как точность

восстановления, так и устойчивость генерации к зашумленным телеметрическим данным.

1.3 Научная новизна

— Разработана оригинальная модификация конвейера вариационного автоэнкодера (TimeSeries Denoising VAE), совмещающая в себе принципы шумоподавления (Denoising) на этапе кодирования предыстории и пошаговый авторегрессионный (Closed-loop) механизм генерации многомерного будущего. В отличие от классических VAE, предсказывающих весь горизонт прогнозирования единой матрицей и приводящих к математическому коллапсу (усреднению траекторий до константных прямых), предложенный подход позволяет рекурсивно моделировать нелинейную физику деградации датчиков.

— Предложена стохастическая схема пошагового сэмплирования динамических априорных распределений (Z_t) в скрытом пространстве рекуррентных и марковских моделей (VRNN, Deep SSM), адаптированная для оценки остаточного ресурса (RUL) авиационных двигателей в условиях неопределенности. Это позволило воссоздавать не просто детерминированный тренд износа, а реалистичные физические колебания параметров (микро-флуктуации датчиков), отражающие реальные условия эксплуатации.

1.4 Теоретическая и практическая значимость

Теоретическая значимость заключается в обосновании применения вариационного автокодировщика для совместного решения задач многошагового прогнозирования и вероятностной оценки надежности. За счет совместной оптимизации точности генерации будущих циклов и регуляризации скрытого пространства дивергенцией Кульбака-Лейблера обеспечен переход от детерминированного прогнозирования к стохастической оценке состояния, позволяющей количественно измерять неопределенность и риск отказа в условиях отсутствия данных об аварийных режимах.» Полученные результаты создают теоретическую основу для построения систем предиктивного обслуживания, способных количественно оценивать неопределенность прогноза и

адаптироваться к изменяющимся условиям эксплуатации без необходимости переобучения на аварийных сценариях.

Практическая значимость работы заключается в разработке открытого программного комплекса в виде Python-библиотеки, реализующего полный жизненный цикл системы предиктивного обслуживания оборудования. Разработанный инструмент обеспечивает следующее:

1. Обеспечение масштабируемости обработки телеметрических данных. За счет использования генераторов при чтении данных реализована конвейерная обработка потоков информации, что позволяет работать с большими массивами телеметрии без полной выгрузки их в оперативную память. Это критически важно для задач мониторинга парков оборудования, где объемы данных измеряются терабайтами.

2. Ускорение внедрения методов машинного обучения. Модульная структура библиотеки, включающая пакеты предобработки и автоматизированные пайплайны, снижает порог входа для инженеров-разработчиков. Стандартизированные процедуры очистки и нормализации данных минимизируют количество ручных операций при подготовке датасетов.

3. Промышленная готовность и воспроизводимость. Интеграция сервисов MLflow позволяет осуществлять версионирование моделей, логирование гиперпараметров экспериментов и централизованное хранение артефактов. Это обеспечивает прозрачность процесса обучения и возможность быстрого отката к предыдущим версиям моделей при изменении условий эксплуатации.

4. Упрощение процесса принятия решений. В состав библиотеки включены специализированные методы (например, автоматический расчет кумулятивной функции распределения ошибок и прогнозирование остаточного ресурса), которые абстрагируют сложные математические вычисления. Это позволяет эксплуатационному персоналу получать интерпретируемые метрики риска отказа без необходимости глубокого понимания внутреннего устройства нейросетевых архитектур.

5. Возможность независимого использования модулей. Архитектура библиотеки спроектирована таким образом, что каждый пакет (чтение данных, метрики, модели) может быть использован автономно или интегрирован в существующие корпоративные системы мониторинга через программный интерфейс (API).

Валидация разработанных алгоритмов и программной реализации выполнена на базе открытого набора данных NASA C-MAPSS. Полученные результаты подтверждают корректность работы модулей прогнозирования и создают основу для переноса методики на задачи мониторинга реального оборудования.

1.5 Положения, выносимые на защиту

Метод предиктивного моделирования деградации сложных технических систем на базе оригинальной модификации вариационного автоэнкодера (TimeSeries Denoising VAE). В отличие от традиционных вероятностных подходов прямого прогнозирования, приводящих к математическому коллапсу дисперсии и аппроксимации будущего плоскими константными линиями, предложенный метод реализует концепцию пошагового авторегрессионного инференса с глубокой обратной связью (Closed-loop forecasting). Это обеспечивает устойчивое воссоздание долгосрочных, сложновыраженных нелинейных траекторий износа многомерных временных рядов (показаний датчиков) с сохранением их физических законов затухания и роста.

1.6 Выводы

В рамках первой главы выполнен анализ предметной области прогнозирования технического состояния сложного оборудования и проведен сравнительный обзор существующих методов машинного обучения. На основе проведенного исследования сформулированы следующие основные результаты:

1. Установлено, что традиционные детерминированные методы прогнозирования не обеспечивают оценки неопределенности прогноза, что критично для задач мониторинга ответственных узлов. Генеративные состязательные сети и диффузионные модели, несмотря на высокую точность генерации, обладают значительной вычислительной сложностью, затрудняющей их применение в системах реального времени.

2. Предложена концепция многошагового прогнозирования на основе амортизированного вывода параметров скрытого распределения. Использование скользящего окна из n предыдущих циклов позволяет модели агрегировать контекст временного ряда, снижая влияние локальных шумов теле-

метрии на точность оценки текущего состояния.

3. Разработан методологический подход к детектированию аномалий без использования размеченных данных об отказах. Суть подхода заключается в обучении модели исключительно на данных нормального функционирования и последующем вычислении вероятности отказа через эмпирическую функцию распределения ошибок реконструкции.

4. Сформулированы функциональные и нефункциональные требования к программной реализации системы. Определена модульная архитектура комплекса, включающая конвейер обработки данных, блок генеративного моделирования и сервисы управления экспериментами, что обеспечивает масштабируемость и воспроизводимость результатов.

Полученные теоретические и методологические результаты создают необходимую базу для перехода к практической реализации. Во второй главе будет выполнено детальное проектирование системы моделирования, формализована математическая постановка задачи и разработана архитектура программного комплекса.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ МОДЕЛИРОВАНИЯ ДАННЫХ

2.1 Постановка задачи

Для создания системы предиктивного моделирования состояния технологического оборудования в качестве базового источника данных принят датасет NASA C-MAPSS (Commercial Modular Aero-Propulsion System Simulation), содержащий телеметрические показания турбовентиляторных двигателей в различных режимах эксплуатации и условиях окружающей среды.

Требуется спроектировать и реализовать программный комплекс для вероятностного моделирования и прогнозирования состояния оборудования на основе нейросетевых генеративных моделей. Основными требованиями, предъявляемыми к системе, являются:

1. Система должна осуществлять многошаговое прогнозирование телеметрических параметров на n эксплуатационных циклов вперёд с оценкой вероятностной неопределённости.
2. Программная реализация должна базироваться на архитектуре вариационного автокодировщика (VAE), обеспечивающей явное вероятностное латентное пространство и устойчивую оптимизацию.
3. Система должна принимать на вход временные ряды показаний датчиков в форматах CSV/Parquet. Единицей обработки выступает скользящее окно из n последовательных циклов $X_{1:n}$.
4. Система должна формировать скалярный индикатор состояния S_t на основе статистик латентного пространства и ошибки реконструкции, включая векторное представление статуса оборудования.
5. Модель должна поддерживать модульную интеграцию: загрузку предобученных весов из удалённого хранилища и экспорт предсказаний в форматы, совместимые с промышленными SCADA/MES-системами.
6. Все программные компоненты должны быть оформлены в виде независимых Python-пакетов с чётким разделением ответственности (предобработка, обучение, инференс, валидация).
7. В системе должна быть реализована автоматизированная регистрация метрик обучения, гиперпараметров и версий датасетов для обеспечения воспроизводимости экспериментов.

8. Оценка качества модели должна производиться по комплексу метрик: поточечная MSE, DTW для временных рядов, а также классификационные показатели на основе порогов S_t .

2.1.1 Описание входных данных

Исходным информационным базисом для проведения исследований является плоский двумерный массив полетной телеметрии, содержащий последовательные записи о состоянии парка авиационных двигателей NASA Turbofans (C-MAPSS). Вектор сырых данных для каждого отдельного двигателя представляет собой временную последовательность полетных циклов.

Пусть $\mathbf{X}_{\text{raw}} \in \mathbb{R}^{N_{\text{total}} \times (2+D)}$ — исходная плоская матрица данных, где $N_{\text{total}} = 86914$ — общее число зарегистрированных полетных циклов (строк) в выборке, а размерность столбцов включает в себя два служебных идентификатора и $D = 26$ информационных каналов (операционные настройки и показания физических датчиков). Спецификация столбцов исходной матрицы имеет вид:

$$\mathbf{X}_{\text{raw}} = [\mathbf{u}, \mathbf{c}, \mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_D], \quad (1)$$

где $\mathbf{u} \in \mathbb{N}^{N_{\text{total}}}$ - вектор идентификаторов контролируемых двигателей (*unit number*), $\mathbf{c} \in \mathbb{N}^{N_{\text{total}}}$ - вектор порядковых номеров полетных циклов (*time in cycles*), а $\mathbf{f}_d \in \mathbb{R}^{N_{\text{total}}}$ - вектор физических измерений d -го информационного канала.

В рамках разработанного конвейера предобработки служебные столбцы ($\mathbf{u}, \mathbf{c}$) изолируются от процедуры внесения искажений, а матрица датчиков подвергается Z-нормализации (*StandardScaler*) и процедуре скользящего сглаживания (*Rolling Mean*) с размером окна в 5 циклов. Это позволяет сформировать эталонную плоскую матрицу чистых физических трендов:

$$\mathbf{X}_{\text{clean}} = \text{StandardScaler}(\text{RollingMean}([\mathbf{f}_1, \dots, \mathbf{f}_D])) \in \mathbb{R}^{N_{\text{total}} \times D}. \quad (2)$$

Для проведения стресс-тестирования моделей на устойчивость к отказам бортового оборудования формируется искаженная матрица входных признаков $\mathbf{X}_{\text{stressed}} = \text{apply_stress_to_data}(\mathbf{X}_{\text{clean}})$. На основе переданного параметра *noise_type* на отмасштабированные датчики накладываются два типа аппаратных дефектов:

К каждому элементу матрицы чистых признаков добавляется случайная составляющая, подчиняющаяся нормальному (Гауссову) закону распределения с нулевым математическим ожиданием и заданным уровнем стандартного отклонения $\sigma_{\text{noise}} = 0.1$:

$$x_{\text{stressed},i,d} = x_{\text{clean},i,d} + \epsilon_{i,d}, \quad \epsilon_{i,d} \sim \mathcal{N}(0, \sigma_{\text{noise}}^2). \quad (3)$$

Симулируется случайная потеря пакетов телеметрии с интенсивностью $\text{drop_rate} = 0.2$. Ровно 20% случайных ячеек матрицы датчиков заменяются на индикатор полного отсутствия сигнала $\text{fill_value} = 0.0$:

$$x_{\text{stressed},i,d} = \begin{cases} 0.0, & \text{с вероятностью } 0.2, \\ x_{\text{clean},i,d}, & \text{с вероятностью } 0.8. \end{cases} \quad (4)$$

В комбинированном режиме оба дефекта накладываются последовательно, после чего значения принудительно ограничиваются методом *clip* для предотвращения математических выбросов.

На финальном этапе плоские матрицы нарезаются на скользящие трехмерные окна с заданной глубиной и значением сдвига. В результате формируются две ключевые структуры данных, поступающие на вход нейросети:

1. Искаженная предыстория (Вход Энкодера): трехмерный тензор $\mathcal{X}_{\text{stressed}} \in \mathbb{R}^{B \times M \times D}$, содержащий зашумленные и рваные сигналы первых 10 циклов скользящего окна. На основе этой матрицы энкодер учится строить инвариантный латентный код стиля.

2. Идеальная предыстория (Точка опоры Декодера): изолированный вектор $\mathbf{x}_{\text{anchor}} \in \mathbb{R}^{B \times D}$, представляющий собой строго крайнее, отмасштабированное значение из сглаженной (чистой) матрицы $\mathbf{X}_{\text{clean}}$. Данный вектор служит безошибочным физическим якорем, от которого декодер рекурсивно вычисляет приращения дельт в будущее.

Здесь $B = 87894$ — результирующее количество сформированных трехмерных окон (размерность батча) после фильтрации граничных циклов индивидуальных двигателей.

2.1.2 Описание выходных данных

Целевым результатом работы системы является генерация вероятностного прогноза будущего состояния оборудования. Выход модели представляет собой последовательность сгенерированных векторов наблюдений для k будущих циклов:

Пусть $D = 26$ — размерность пространства измеряемых признаков (каналов датчиков), $M = 10$ — глубина анализируемой предыстории (длина входного зашумленного окна), а $T = 20$ — полный временной горизонт, включающий фазу восстановления и фазу прогнозирования.

Каждый s -й сгенерированный моделью сценарий представляет собой многомерный временной ряд, описываемый матрицей $\hat{\mathbf{Y}}^{(s)}$ размерности $(T \times D)$:

$$\hat{\mathbf{Y}}^{(s)} = \begin{bmatrix} \hat{y}_{1,1}^{(s)} & \hat{y}_{1,2}^{(s)} & \cdots & \hat{y}_{1,D}^{(s)} \\ \hat{y}_{2,1}^{(s)} & \hat{y}_{2,2}^{(s)} & \cdots & \hat{y}_{2,D}^{(s)} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{y}_{T,1}^{(s)} & \hat{y}_{T,2}^{(s)} & \cdots & \hat{y}_{T,D}^{(s)} \end{bmatrix} \in \mathbb{R}^{T \times D}, \quad s \in \{1, 2, \dots, S\}, \quad (5)$$

где $S = 5$ — общее количество генерируемых альтернативных сценариев для оценки интервальной неопределенности, а $\hat{y}_{t,d}^{(s)}$ — нормализованное (StandardScaler) значение d -го датчика в момент времени t в рамках s -го сценария.

Структурно выходная матрица $\hat{\mathbf{Y}}^{(s)}$ разделена на две функциональные зоны вдоль временной оси t :

1. Зона деноизинга и реконструкции ($1 \leq t \leq M$): фаза, на которой декодер восстанавливает истинные значения на основе латентного вектора стиля $\mathbf{z}^{(s)}$. В силу использования Denoising-подхода, выходные значения $\hat{y}_{t,d}^{(s)}$ аппроксимируют сглаженный (Rolling Mean) физический тренд деградации $\mathbf{Y}_{\text{smooth}}$, полностью игнорируя наложенный на входные данные $\mathbf{X}_{\text{stressed}}$ измерительный хаос и пропуски:

$$\hat{\mathbf{Y}}_{1:M}^{(s)} \approx \mathbf{Y}_{\text{smooth},1:M}, \quad \text{при этом} \quad \hat{\mathbf{Y}}_{1:M}^{(1)} \equiv \hat{\mathbf{Y}}_{1:M}^{(2)} \equiv \cdots \equiv \hat{\mathbf{Y}}_{1:M}^{(S)}. \quad (6)$$

Траектории всех сценариев в этой зоне идентичны, так как они жестко привязаны к детерминированной предыстории конкретного двигателя.

2. Зона авторегрессионного прогноза ($M < t \leq T$): фаза автономной

генерации будущего с глубокой обратной связью (Closed-loop). Значение на шаге t формируется рекурсивно на основе вектора приращений (дельта) $\Delta_t^{(s)}$, вычисляемого моделью на базе накопленной предыстории скрытых состояний:

$$\hat{\mathbf{y}}_t^{(s)} = \hat{\mathbf{y}}_{t-1}^{(s)} + \Delta_t^{(s)} \left(\hat{\mathbf{y}}_{1:t-1}^{(s)}, \mathbf{z}_t^{(s)} \right), \quad t \in \{M+1, \dots, T\}. \quad (7)$$

Для получения финального точечного прогноза и оценки дисперсии (интервальной неопределенности) пооконных траекторий на диссертационную защиту выносятся использование операторов математического ожидания $\mathbb{E}$ и дисперсии $\mathbb{D}$ над множеством сценариев S :

$$\bar{\mathbf{y}}_t = \mathbb{E}_s \left[\hat{\mathbf{y}}_t^{(s)} \right] = \frac{1}{S} \sum_{s=1}^S \hat{\mathbf{y}}_t^{(s)}, \quad (8)$$

$$\sigma_t^2 = \mathbb{D}_s \left[\hat{\mathbf{y}}_t^{(s)} \right] = \frac{1}{S-1} \sum_{s=1}^S \left(\hat{\mathbf{y}}_t^{(s)} - \bar{\mathbf{y}}_t \right)^2. \quad (9)$$

Величина $\sigma_t^2 \in \mathbb{R}^D$ описывает расхождение веера траекторий. Рост дисперсии во времени ($\sigma_{t_2}^2 > \sigma_{t_1}^2$ при $t_2 > t_1 > M$) отражает экспоненциальное накопление неопределенности прогноза остаточного ресурса (RUL) по мере удаления от последней известной физической точки отсчета технического состояния авиадвигателя.

3 ПРОВЕДЕНИЕ ЭКСПЕРИМЕНТОВ И АНАЛИЗ РЕЗУЛЬТАТОВ

3.1 Описание исследуемого набора данных и предварительная обработка

3.2 Архитектура экспериментальных моделей и программная реализация

3.3 Методология и протокол обучения моделей

3.4 Система метрик и методы стресс-тестирования

3.5 Анализ и интерпретация экспериментальных результатов

3.6 Выводы

4 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ МОДЕЛИРОВАНИЯ

4.1 Архитектура программного комплекса

Архитектура программного комплекса должна представлять собой модульную систему, реализующую полный жизненный цикл предиктивного обслуживания технологического оборудования. Архитектура построена по принципу разделения ответственности и включает четыре функциональных слоя: слой данных, слой предобработки, слой моделирования и слой метрик. Интеграция с платформой MLflow обеспечивает версионирование артефактов и воспроизводимость экспериментов.

Слой данных отвечает за хранение и управление входными потоками телеметрии. Он включает хранилище сырых данных, содержащих показания датчиков в исходном формате, и хранилище предобработанных данных, куда записываются нормализованные временные ряды, готовые к обучению. Разделение на два хранилища позволяет избежать повторной обработки данных при многократных запусках экспериментов и ускоряет итерации разработки

моделей.

Слой предобработки реализует конвейер трансформации данных. Модуль загрузки данных (Data Loader) обеспечивает потоковое чтение файлов без полной выгрузки в оперативную память, что критически важно для работы с большими массивами телеметрии. Модуль очистки (Data Cleaner) выполняет интерполяцию пропущенных значений и фильтрацию высокочастотных шумов. Модуль нормализации (Normalizer) применяет Z-score стандартизацию для приведения разнородных датчиков к единому масштабу. Генератор окон (Window Generator) агрегирует последовательные циклы в тензоры фиксированной размерности $X_{1:n}$, формируя входные данные для обучения.

Слой моделирования содержит ядро системы — вариационный автокодировщик. Кодировщик (Encoder) реализует амортизированный вывод параметров латентного распределения $q_\phi(z \mid X_{1:n}) = \mathcal{N}(z; \mu_\phi, \text{diag}(\sigma_\phi^2))$. Латентное пространство (Latent Space) обеспечивает хранение сжатого стохастического представления состояния оборудования. Декодировщик (Decoder) генерирует последовательность будущих циклов $\hat{X}_{n+1:n+k}$ на основе сэмплированного вектора z . Детектор аномалий (Anomaly Detector) вычисляет скалярный индикатор состояния S_t и вероятность отказа на основе эмпирической функции распределения ошибок реконструкции.

Слой метрик отвечает за количественную оценку качества моделирования. Калькулятор метрик (Metrics Calculator) вычисляет поточечные ошибки (RMSE, MAE) и структурные показатели (DTW) для сгенерированных траекторий. Калибратор порогов (Threshold Calibrator) определяет граничные значения индикатора S_t на основе распределения ошибок на обучающей выборке. Предиктор остаточного ресурса (RUL Predictor) экстраполирует тренд индикатора состояния и оценивает время до пересечения критического порога.

Интеграция с MLflow реализована через два компонента. Реестр моделей (Model Registry) хранит веса обученных нейросетей, параметры калибровки порогов и метаданные экспериментов. Трекер экспериментов (Exp Tracker) фиксирует значения метрик, гиперпараметры и версии данных, обеспечивая полную воспроизводимость исследований.

Архитектура поддерживает три основных сценария функционирования. Сценарий обучения (обозначен сплошными линиями на рисунке) вклю-

чает загрузку сырых данных, их предобработку, оптимизацию функции ELBO и сохранение модели в реестр. Сценарий инференса (пунктирные линии) выполняет загрузку новой порции телеметрии, генерацию прогноза и вычисление вероятности отказа с использованием сохранённой модели. Сценарий дообучения (штрихпунктирные линии) позволяет адаптировать модель к изменяющимся условиям эксплуатации путём тонкой настройки весов на новых данных без полного переобучения.

Подобная модульная организация обеспечивает гибкость масштабирования, независимое обновление компонентов и возможность интеграции с промышленными системами мониторинга. Детальная структура программных пакетов и описание их взаимодействия будут приведены в следующих разделах.

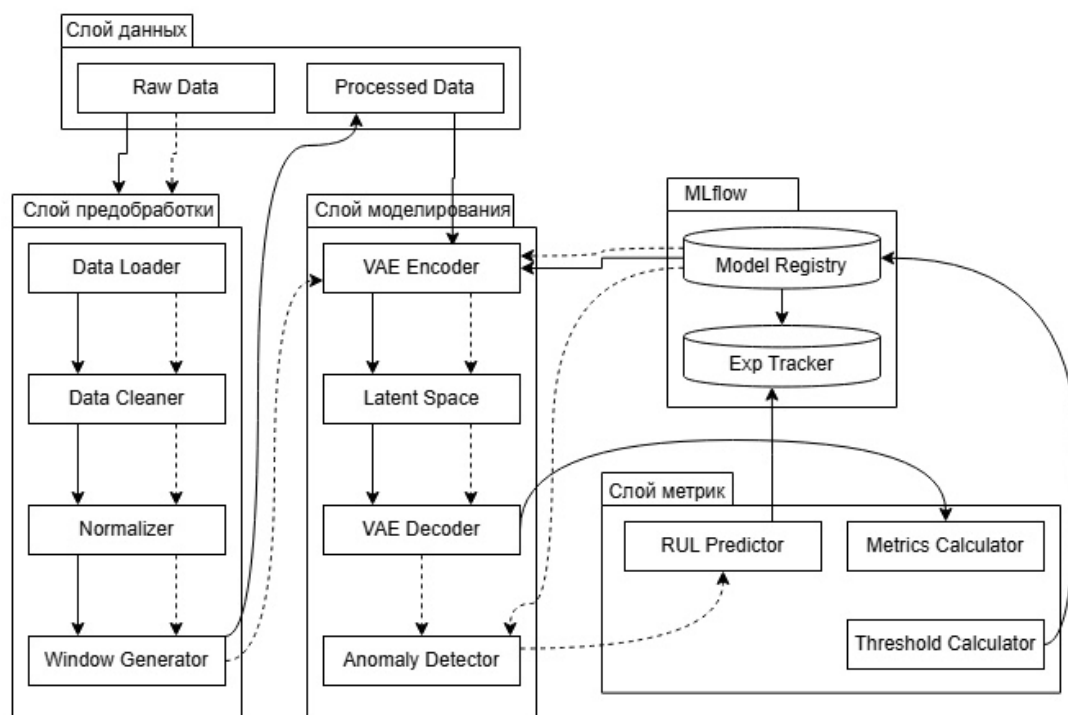


Рисунок 1 — Архитектура системы обучения и прогнозирования

3.2 Структура Python-библиотеки и организация пакетов

Программный комплекс реализован в виде открытой Python-библиотеки с модульной архитектурой, обеспечивающей четкое разделение ответствен-

ности между компонентами. Структура проекта построена по принципу независимых пакетов, что позволяет использовать каждый модуль автономно или в составе единого конвейера обработки данных. Корневая директория библиотеки содержит конфигурационные файлы, документацию и исходный код, организованный в следующие пакеты:

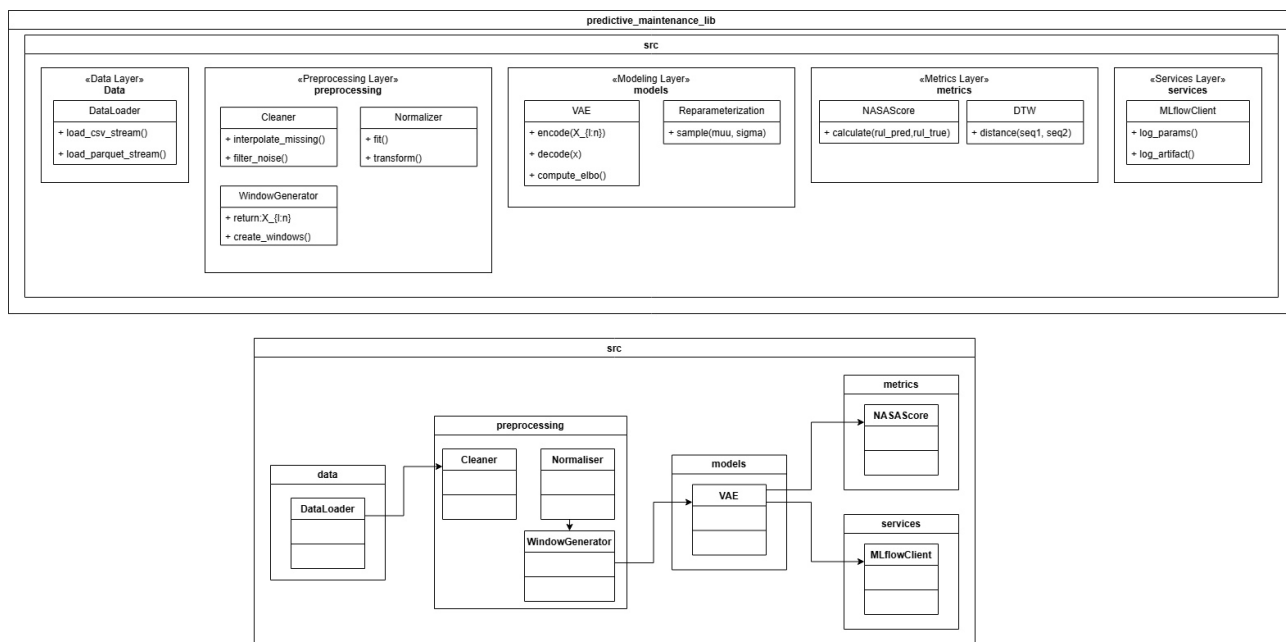


Рисунок 2 — Структура пакетов Python-библиотеки

Пакет данных отвечает за загрузку телеметрических рядов из внешних источников. Реализация основана на использовании итераторов и генераторов, что обеспечивает конвейерную обработку потоков информации без полной выгрузки массивов в оперативную память. Поддерживаются форматы CSV и Parquet, а также механизмы кэширования часто используемых сегментов временных рядов.

Пакет предобработки содержит процедуры фильтрации высокочастотных шумов, интерполяции пропущенных значений и синхронизации временных меток. Модуль нормализации применяет Z-score стандартизацию на основе статистик обучающей выборки, что гарантирует согласованность масштабов разнородных датчиков. Генератор скользящих окон преобразует непрерывные временные ряды в тензоры фиксированной размерности $X_{1:n}$, готовые для подачи в нейросетевую архитектуру.

Пакет пайплайна выполняет функцию оркестратора, координируя последовательность вызовов модулей чтения, трансформации и обучения. Кон-

фигурационный менеджер позволяет задавать параметры обработки через внешние YAML-файлы, что упрощает адаптацию системы к новым типам оборудования без изменения исходного кода. Логгер событий фиксирует этапы выполнения конвейера и сохраняет метаданные о версиях используемых данных.

Пакет моделей содержит реализацию вариационного автокодировщика, включая классы кодировщика, декодировщика и вероятностного слоя репараметризации. Архитектура поддерживает как полносвязные, так и рекуррентные варианты скрытых блоков. Методы класса предоставляют функции прямого прохождения для генерации последовательностей $\hat{X}_{n+1:n+k}$ и вычисления параметров апостериорного распределения латентных переменных.

Пакет метрик объединяет функции вычисления поточечных ошибок, структурных показателей временных рядов и специализированных критериев для оценки остаточного ресурса. Реализованы расчеты RMSE, MAE, DTW, а также функция потерь NASA Score. Модуль статистической валидации содержит процедуры для попарного сравнения моделей и проверки значимости различий в показателях качества.

Пакет сервисов обеспечивает интеграцию с платформой управления экспериментами. Клиентский модуль реализует методы логирования гиперпараметров, сохранения артефактов обучения в централизованное хранилище и загрузки версий моделей для инференса. Автоматическое версионирование артефактов гарантирует воспроизводимость результатов при повторных запусках экспериментов.

Установка библиотеки выполняется через стандартный менеджер пакетов Python после клонирования репозитория. Зависимости проекта изолированы в виртуальном окружении, что предотвращает конфликты версий библиотек при интеграции с другими аналитическими системами. Структура исходного кода сопровождается модульными тестами, проверяющими корректность обработки граничных условий и согласованность размерностей тензоров на этапах конвейера.

Детальное описание функциональных возможностей каждого модуля и алгоритмов их взаимодействия будет приведено в следующем разделе.

3.3 Описание функциональных модулей

3.3.1 Модуль чтения и обработки данных

Модуль чтения и обработки данных реализует пакет `src.data` и отвечает за первичное взаимодействие с источниками телеметрии. В контексте работы с датасетом NASA C-MAPSS, содержащим длинные временные ряды работы двигателей, критически важным аспектом является оптимизация использования оперативной памяти. Для этого реализован механизм ленивой загрузки данных с использованием генераторов Python. Вместо полной выгрузки массива в память, система последовательно считывает фрагменты файлов, обрабатывает их и передает в конвейер обучения, что позволяет работать с датасетами объемом в гигабайты на оборудовании с ограниченными ресурсами.

Особенностью предобработки для вариационных автокодировщиков является чувствительность модели к масштабу входных данных и распределению признаков. Процесс трансформации включает три ключевых этапа:

Первый этап — очистка и восстановление данных. Сырые телеметрические сигналы могут содержать пропуски (NaN) и высокочастотные выбросы, вызванные сбоями сенсоров. Модуль применяет линейную интерполяцию для заполнения пропусков на основе соседних циклов. Для сглаживания случайных шумов, не несущих информации о деградации, используется скользящее среднее с окном малого размера. Это предотвращает обучение VAE на артефактах измерений, заставляя модель фокусироваться на физических процессах износа.

Второй этап — стандартизация (Z-score normalization). Поскольку VAE обучается путем минимизации функции потерь, включающей евклидову норму ошибки реконструкции, признаки с большим диапазоном значений (например, давление) могут доминировать над признаками с малым диапазоном (например, температура). Модуль вычисляет математическое ожидание и стандартное отклонение для каждого из d каналов датчиков. Важно отметить, что статистики вычисляются исключительно на обучающей выборке (штатный режим), чтобы избежать утечки информации о будущих отказах в процесс нормализации.

Третий этап — формирование скользящих окон. Вариационный авто-

кодировщик требует на входе тензоры фиксированной размерности. Модуль WindowGenerator преобразует непрерывный временной ряд длиной T_m в набор перекрывающихся окон фиксированной длины n . Каждое окно представляет собой матрицу $X_{t-n+1:t} \in \mathbb{R}^{n \times d}$, где строки соответствуют последовательным циклам, а столбцы — показаниям датчиков.

Для задачи обучения генеративной модели на данных нормы модуль предоставляет опцию фильтрации: из полного жизненного цикла двигателя автоматически выбираются только первые n_{train} циклов. Это обеспечивает соответствие теоретической постановке задачи, где модель должна изучить распределение только исправного состояния оборудования. Результирующий поток данных представляет собой итератор тензоров, готовых к передаче в слой кодировщика нейросетевой архитектуры.

Структура классов модуля чтения и обработки данных представлена на рисунке.

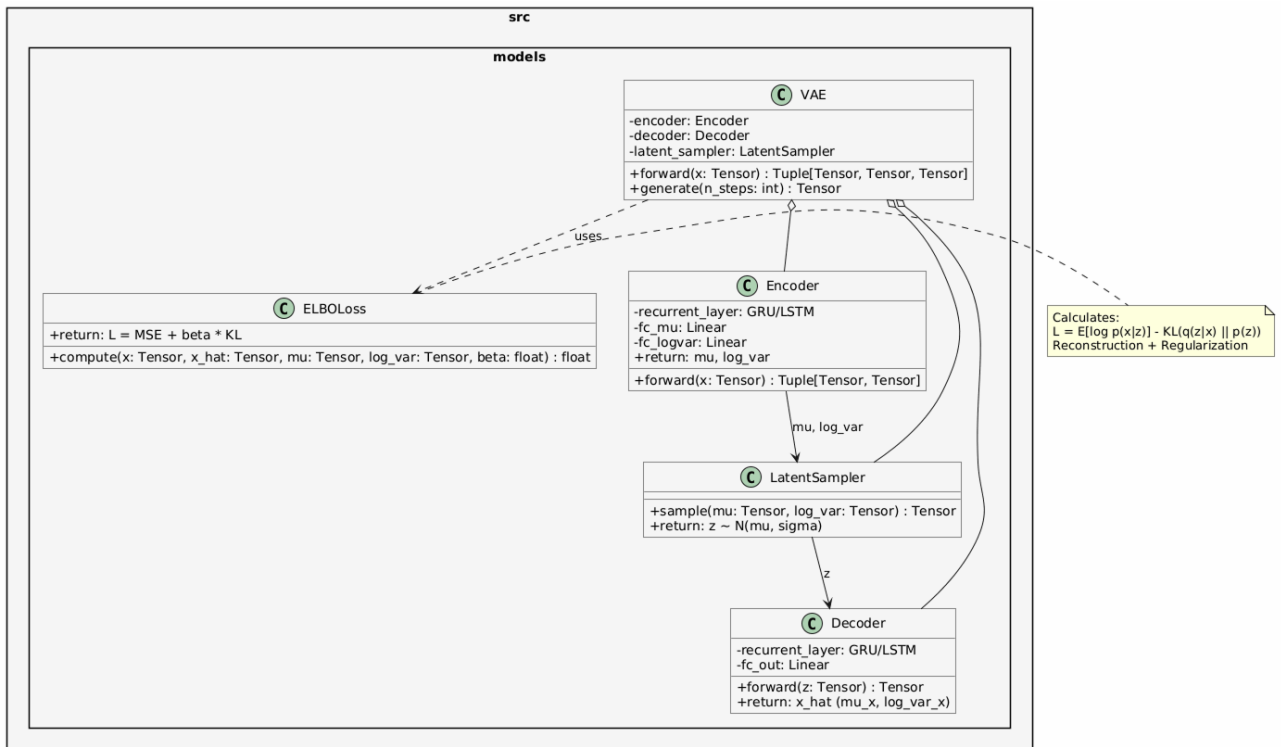


Рисунок 3 — Диаграмма классов модуля чтения и обработки данных

Основой модуля является класс DataLoader, реализующий паттерн итератора. Метод stream_data обеспечивает последовательное чтение исходных файлов небольшими порциями (чанками). Это позволяет избежать загрузки полного датасета в оперативную память, что критически важно для обработки длительных временных рядов телеметрии двигателей.

Класс `DataCleaner` инкапсулирует логику очистки сигналов. Метод `interpolate_missing` восстанавливает пропущенные значения с помощью линейной интерполяции, а `filter_noise` применяет сглаживающие фильтры для подавления высокочастотных шумов сенсоров.

Особое значение для обучения вариационного автокодировщика имеет класс `DataNormalizer`. Поскольку VAE использует евклидову метрику в функции потерь, признаки с большими абсолютными значениями могут доминировать над признаками с малыми значениями, что приводит к неустойчивому обучению. Метод `fit` вычисляет математическое ожидание и среднеквадратичное отклонение для каждого канала датчиков. Ключевым требованием является то, что эти статистики вычисляются исключительно на данных штатного режима (обучающая выборка), чтобы предотвратить утечку информации о состояниях деградации в процесс нормализации. Метод `transform` применяет Z-score стандартизацию к поступающим данным.

Класс `WindowGenerator` отвечает за формирование входных тензоров для нейросети. Метод `create_windows` преобразует нормализованный временной ряд в последовательность перекрывающихся окон фиксированной длины n . Результатом работы метода является тензор размерности $n \times d$, соответствующий обозначению $X_{1:n}$, используемому в математической постановке задачи.

Координацию работы всех компонентов выполняет класс `DataPipeline`. Метод `execute` инициализирует цепочку вызовов: загрузка $\rightarrow$ очистка $\rightarrow$ нормализация $\rightarrow$ формирование окон. Данный класс возвращает генератор, который по требованию выдает готовые батчи данных для процедуры обучения модели, обеспечивая непрерывный поток информации без простоев вычислительных ресурсов.

3.3.2 Модуль предобработки и пайплайна

3.3.2 Модуль предобработки и пайплайна

Модуль пайплайна реализует пакет `src.pipeline` и выполняет функцию оркестратора, координируя последовательность вызовов компонентов чтения, трансформации и обучения. В отличие от модуля данных, отвечающего за непосредственную обработку сигналов, данный пакет управляет состояни-

ем эксперимента, валидацией входных параметров и логированием событий. Структура классов модуля представлена на рисунке.

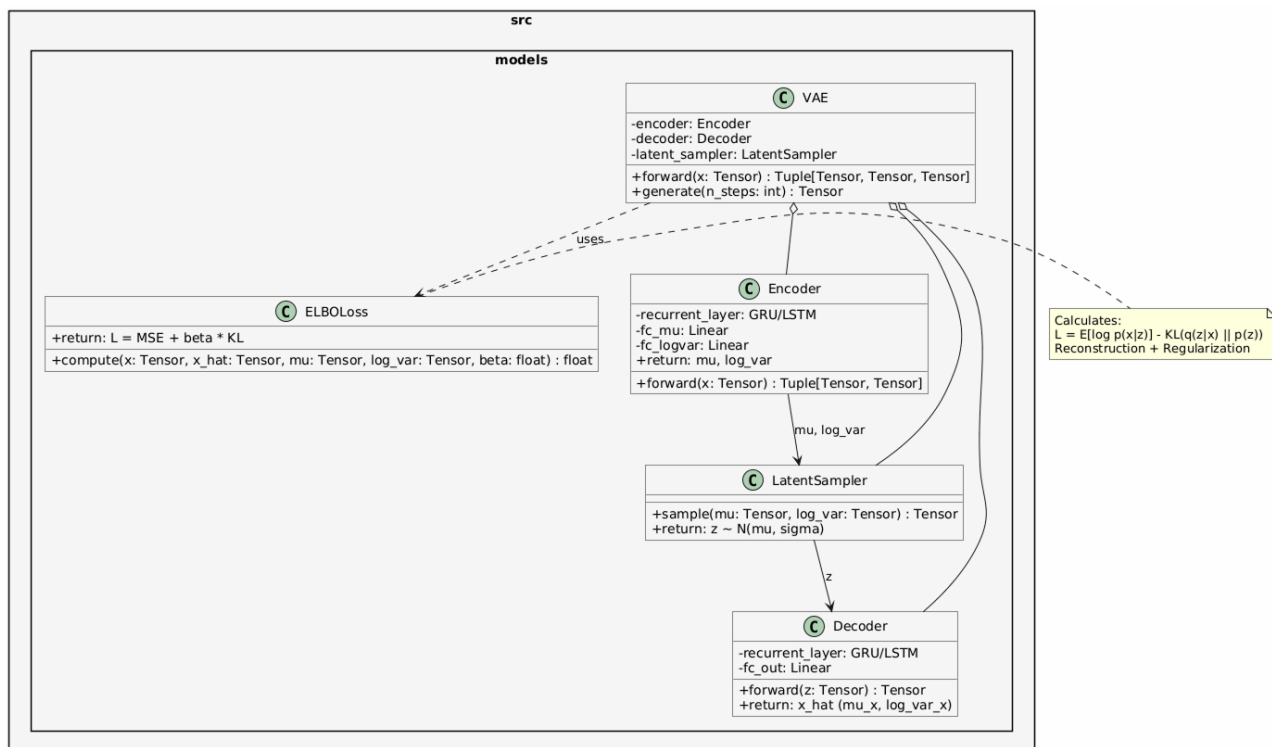


Рисунок 4 — Диаграмма классов модуля пайплайна

Класс ConfigManager обеспечивает внешнее управление гиперпараметрами системы. Вместо жесткого кодирования значений в исходном коде, все настройки, включая длину окна n , горизонт прогнозирования k , коэффициенты регуляризации λ и пути к хранилищам, вынесены в YAML-файлы конфигурации. Метод `load_yaml` парсит конфигурационный файл, а `override_params` позволяет программно переопределять параметры во время запуска, что необходимо для автоматизированного перебора сетки гиперпараметров.

Класс DataValidator отвечает за контроль целостности данных перед их подачей в нейросетевую архитектуру. Вариационные автокодировщики чувствительны к аномалиям во входных распределениях, поэтому метод `check_nan_ratio` проверяет долю пропущенных значений после интерполяции. Метод `validate_shape` гарантирует, что тензоры соответствуют ожидаемой размерности $n \times d$. Дополнительно метод `assert_normal_distribution` проверяет, что статистики нормализации вычислены на репрезентативной выборке штатного режима, предотвращая смещение распределения входных признаков.

Класс `ExecutionLogger` интегрирует процесс выполнения пайплайна с платформой управления экспериментами. Метод `log_step` фиксирует начало и завершение каждого этапа обработки, а `log_artifact` сохраняет промежуточные артефакты, такие как вычисленные матрицы нормализации и калибровочные пороги индикатора S_t . Это обеспечивает полную трассировку эксперимента: по сохраненным логам можно точно воспроизвести состояние системы на любом этапе обучения.

Центральным элементом модуля является класс `PipelineOrchestrator`. Метод `run_training_pipeline` инициализирует цепочку выполнения: загрузка конфигурации $\rightarrow$ валидация параметров $\rightarrow$ создание потока данных $\rightarrow$ запуск оптимизации ELBO $\rightarrow$ сохранение артефактов. В случае обнаружения невалидных данных или нарушения контрактов интерфейсов пайплайн немедленно останавливается с генерацией диагностического отчета, что предотвращает обучение модели на некорректных данных и экономит вычислительные ресурсы.

Подобная архитектура пайплайна обеспечивает высокую надежность системы, гибкость настройки экспериментов и соответствие требованиям к воспроизводимости научных результатов. Детальное описание алгоритмов работы генеративных моделей будет приведено в следующем разделе.

3.3.3 Модуль нейросетевых моделей

Модуль генеративных моделей реализует пакет `src.models` и содержит программную реализацию архитектуры вариационного автокодировщика (VAE), адаптированной для работы с временными рядами телеметрии. Данный модуль является ядром системы, обеспечивающим как обучение модели на данных штатного режима, так и генерацию прогнозов состояния оборудования. Структура классов модуля представлена на рисунке.

Центральным классом является VAE, который агрегирует компоненты кодировщика, декодировщика и механизма сэмплирования. Метод `forward` реализует полный прямой проход модели, принимая на вход тензор окна наблюдений $X_{1:n}$ и возвращая сгенерированную последовательность, а также параметры латентного распределения, необходимые для расчета скалярного индикатора S_t .

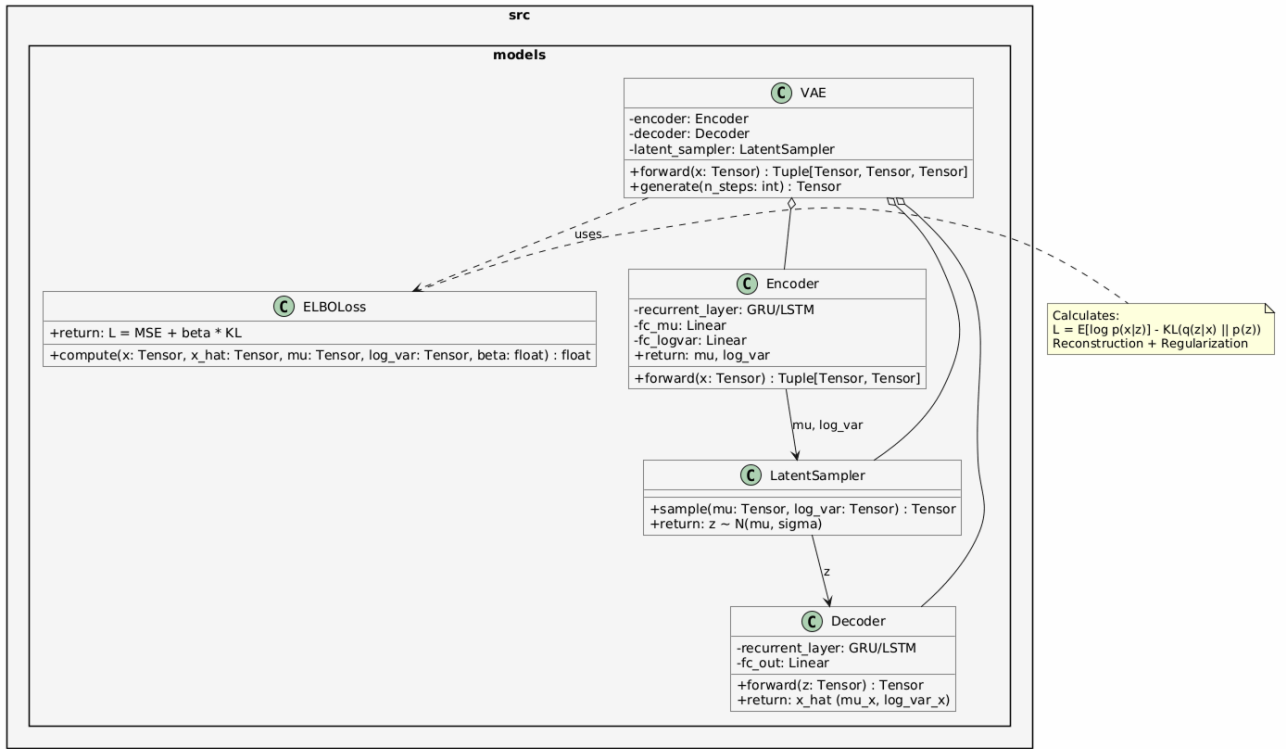


Рисунок 5 — Диаграмма классов модуля нейросетевых моделей

Класс `Encoder` отвечает за аппроксимацию апостериорного распределения $q_\phi(z \mid X_{1:n})$. В соответствии с требованиями к обработке временных рядов, в качестве базового слоя используются рекуррентные блоки (GRU или LSTM), которые последовательно обрабатывают входное окно и агрегируют контекст в скрытом состоянии. Два полносвязных слоя (`fc_mu` и `fc_logvar`) преобразуют выход рекуррентной сети в параметры гауссовского распределения: вектор математического ожидания μ_ϕ и логарифм дисперсии $\log \sigma_\phi^2$. Использование логарифма дисперсии обусловлено соображениями численной устойчивости при оптимизации.

Класс `LatentSampler` реализует трюк репараметризации, необходимый для возможности обратного распространения ошибки через стохастический узел. Метод `sample` генерирует латентный вектор z путем сэмплирования из стандартного нормального распределения $\epsilon \sim \mathcal{N}(0, I)$ и последующего масштабирования: $z = \mu_\phi + \sigma_\phi \cdot \epsilon$. Это позволяет сохранить непрерывность градиентов и обучать параметры распределения совместно с весами нейросети.

Класс `Decoder` реализует вероятностную модель $p_\theta(X_{n+1:n+k} \mid z)$. Он принимает латентный вектор z в качестве условия и генерирует параметры распределения для будущих циклов. Архитектура декодировщика зеркальна кодировщику и также использует рекуррентные слои для развертывания

скрытого состояния в последовательность прогнозов $\hat{X}_{n+1:n+k}$.

Для обучения модели разработан класс `ELBOLoss`, вычисляющий вариационную нижнюю оценку. Метод `compute` объединяет два слагаемых функции потерь: член реконструкции (ошибка между входным окном и его восстановленной версией или предсказанным будущим) и член регуляризации (дивергенция Кульбака-Лейблера между апостериорным и априорным распределениями). Коэффициент β (параметр λ из математической постановки) позволяет балансировать между точностью генерации и структурированностью латентного пространства, что критически важно для последующего детектирования аномалий.

Программная реализация модуля обеспечивает строгое соответствие математическому аппарату, описанному в Главе 2, и предоставляет гибкий интерфейс для настройки архитектуры скрытых слоев в зависимости от сложности телеметрических данных. Описание алгоритмов оценки качества полученных моделей будет приведено в следующем разделе.

3.3.4 Модуль оценки метрик

Модуль оценки метрик реализует пакет `src.metrics` и предоставляет унифицированный интерфейс для количественной оценки качества работы генеративной модели. В отличие от стандартных задач регрессии, оценка вариационного автокодировщика требует комплексного подхода, учитывающего точность поточечного прогнозирования, структурное соответствие временных рядов, качество детектирования аномалий и вероятностную согласованность модели. Структура классов модуля представлена на рисунке.

Класс `RegressionMetrics` реализует базовые метрики точности прогнозирования. Метод `compute_rmse` вычисляет среднеквадратичную ошибку между сгенерированными траекториями и эталонными значениями, чувствительную к крупным отклонениям. Метод `compute_mae` рассчитывает среднюю абсолютную ошибку, обладающую устойчивостью к редким выбросам. Метод `compute_r2` определяет коэффициент детерминации, отражающий долю дисперсии телеметрических параметров, объясненную моделью.

Класс `TemporalMetrics` отвечает за оценку структурного соответствия временных рядов. Метод `compute_dtw` реализует алгоритм динамической

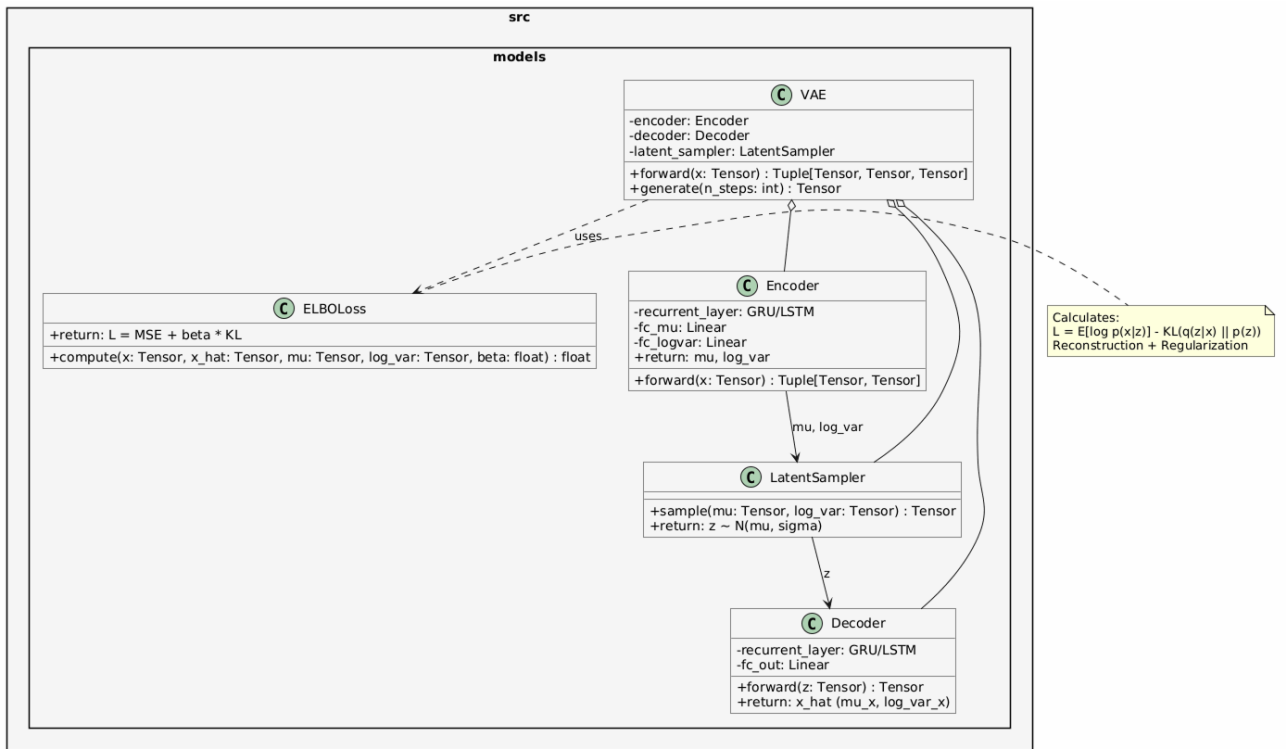


Рисунок 6 — Диаграмма классов модуля оценки метрик

трансформации времени (Dynamic Time Warping), который измеряет минимальное расстояние между двумя последовательностями с учетом нелинейных фазовых сдвигов. Это критически важно для сравнения траекторий деградации, которые могут развиваться с различной скоростью в разных двигателях. Метод `normalize_dtw` выполняет нормализацию сырого значения метрики с учетом длины последовательности, обеспечивая сопоставимость результатов для двигателей с разной продолжительностью жизненного цикла.

Класс `ClassificationMetrics` обеспечивает оценку качества детектирования аномалий на основе скалярного индикатора состояния S_t . Метод `compute_f1` вычисляет гармоническое среднее точности и полноты классификации при заданном пороговом значении. Метод `compute_auc_roc` рассчитывает площадь под ROC-кривой, характеризующую способность модели разделять штатные и аномальные режимы независимо от выбора конкретного порога. Метод `find_optimal_threshold` автоматически определяет граничное значение индикатора S_t , максимизирующее метрику F1 на валидационной выборке.

Класс `NASAScore` реализует специализированную функцию потерь, принятую в датасете NASA C-MAPSS для оценки прогнозирования остаточного ресурса. Метод `compute` применяет асимметричную штрафную функ-

цию: запоздалые прогнозы отказа (когда модель предсказывает отказ позже фактического) штрафуются строже, чем преждевременные, поскольку первые несут большие риски для безопасности эксплуатации. Параметры `penalty_early` и `penalty_late` соответствуют коэффициентам 13 и 10 из оригинальной формулировке метрики.

Класс `ProbabilisticMetrics` предназначен для оценки качества собственно генеративной модели. Метод `compute_elbo` вычисляет значение вариационной нижней оценки на валидационной выборке, позволяя контролировать сходимость процесса обучения. Метод `compute_kl_divergence` измеряет степень соответствия апостериорного распределения латентных переменных априорному стандартному нормальному распределению, что важно для обеспечения непрерывности и интерпретируемости скрытого пространства. Метод `compute_nll` рассчитывает отрицательное логарифмическое правдоподобие, предоставляя альтернативную метрику качества вероятностной генерации.

Класс `StatisticalValidator` обеспечивает статистическую проверку значимости различий между архитектурами моделей. Метод `wilcoxon_test` реализует непараметрический критерий Уилкоксона для связанных выборок, позволяющий сравнивать метрики двух моделей на одних и тех же тестовых двигателях без предположения о нормальном распределении ошибок. Метод `compute_confidence_interval` строит доверительный интервал для среднего значения метрики с заданным уровнем значимости. Метод `compute_effect_size` вычисляет размер эффекта (коэффициент Коэна d) для оценки практической значимости наблюдаемых различий.

Подобная модульная организация метрик позволяет гибко комбинировать критерии оценки в зависимости от целей конкретного эксперимента и обеспечивает объективное сравнение разработанных решений с существующими аналогами. Описание алгоритмов интеграции с платформой управления экспериментами будет приведено в следующем разделе.

3.3.5 Модуль управления экспериментами

Модуль управления экспериментами реализует пакет `src.services` и обеспечивает интеграцию системы с платформой MLflow. Данный модуль реша-

ет задачу автоматизации учета результатов исследования, версионирования обученных моделей и обеспечения воспроизводимости экспериментов. В условиях, когда процесс настройки гиперпараметров требует множества запусков, централизованное логирование позволяет отслеживать влияние изменений архитектуры на качество прогнозирования. Структура классов модуля представлена на рисунке.

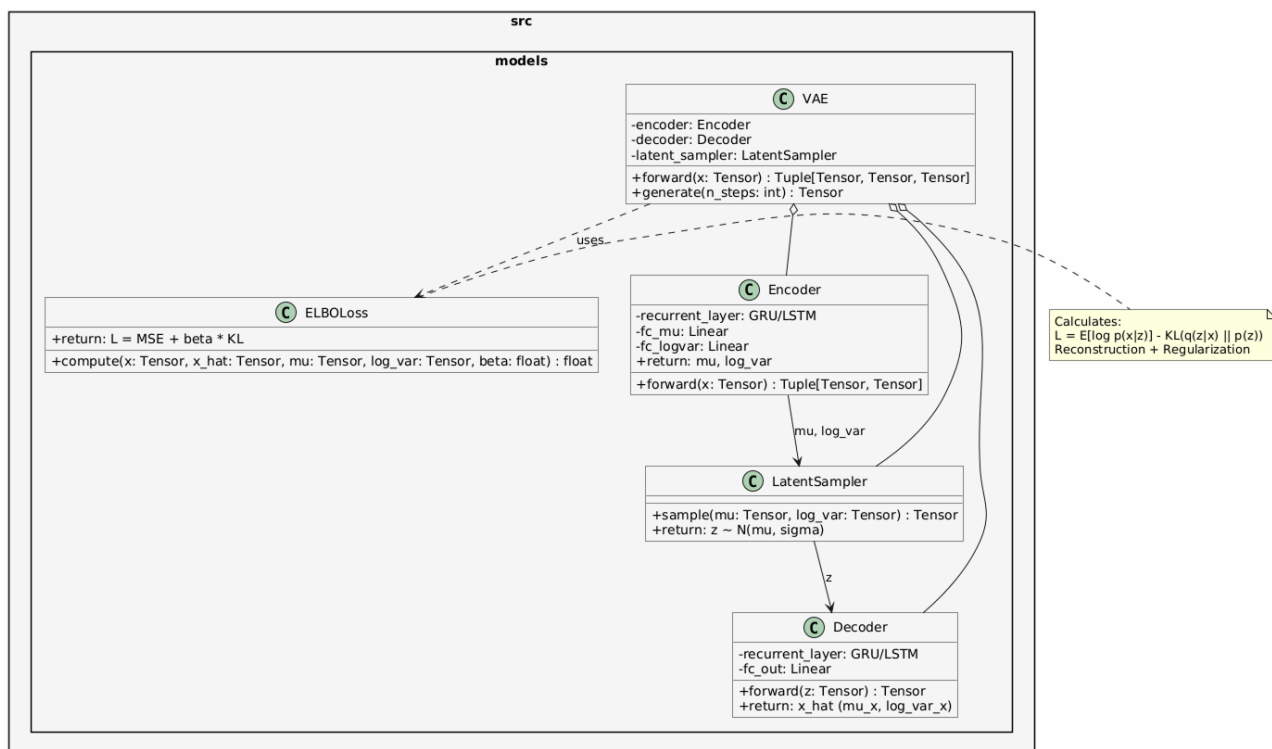


Рисунок 7 — Диаграмма классов модуля управления экспериментами

Класс `MLflowClient` выступает в роли адаптера, инкапсулирующего логику подключения к серверу отслеживания. Метод `init_connection` устанавливает соединение с удаленным хранилищем метаданных, а `set_experiment` определяет логическую группу запусков, соответствующую конкретной серии экспериментов (например, подбор длины окна n или коэффициента регуляризации λ). Это позволяет изолировать результаты различных этапов исследования друг от друга.

Класс `ExperimentTracker` отвечает за фиксацию параметров запуска и динамических метрик. Метод `start_run` инициализирует новый эксперимент и присваивает ему уникальный идентификатор. Метод `log_params` записывает в базу данных значения гиперпараметров модели, такие как размерность латентного пространства, архитектура рекуррентных слоев и параметры оптимизатора. Метод `log_metrics` обеспечивает пошаговую запись значений функ-

ции потерь ELBO и метрик качества (RMSE, DTW) в процессе обучения, что позволяет строить графики сходимости в режиме реального времени.

Класс `ModelRegistry` управляет жизненным циклом артефактов обучения. Метод `log_model` сохраняет сериализованные веса нейросетевой модели вместе с метаданными окружения (версии библиотек, зависимости) в централизованное хранилище. Каждой сохраненной модели присваивается имя и версия, что позволяет однозначно идентифицировать состояние системы на момент обучения. Метод `load_model` обеспечивает загрузку конкретной версии модели для проведения инференса или дообучения, гарантируя, что в рабочем контуре используется проверенная конфигурация.

Класс `ArtifactManager` расширяет функциональность логирования, позволяя сохранять произвольные файлы, такие как конфигурационные YAML-файлы, скрипты предобработки данных и визуализации результатов. Метод `log_artifact` загружает файлы в хранилище, привязывая их к текущему запущенному эксперименту. Это обеспечивает полную трассировку: любой сохраненный результат можно сопоставить с точным набором входных данных и кодом, который его сгенерировал.

Интеграция данного модуля с пайплайном обучения автоматизирует процесс документирования исследования. Все промежуточные результаты, включая калибровочные пороги индикатора S_t и статистики нормализации, сохраняются вместе с моделью, что исключает потерю контекста и упрощает развертывание системы в промышленную эксплуатацию.

Завершение описания программных модулей позволяет перейти к рассмотрению конкретных этапов функционирования системы и сценариев взаимодействия компонентов.

3.5 Направления и возможности функционирования системы

Разработанный программный комплекс поддерживает три основных направления функционирования, соответствующих жизненному циклу предиктивного обслуживания технологического оборудования. Каждое направление реализовано в виде независимого программного пайплайна, использующего общие модули обработки данных, моделирования и логирования. Подобная организация обеспечивает гибкость развертывания системы в различных

условиях эксплуатации и позволяет адаптировать конфигурацию под требования конкретных промышленных объектов.

3.5.1 Пайплайн компонентов

Взаимодействие программных модулей в рамках единого конвейера обработки данных регламентировано классом-оркестратором. Последовательность вызовов компонентов обеспечивает строгую направленность потоков информации и предотвращает циклические зависимости. Логика выполнения пайплайна представлена на диаграмме последовательности.

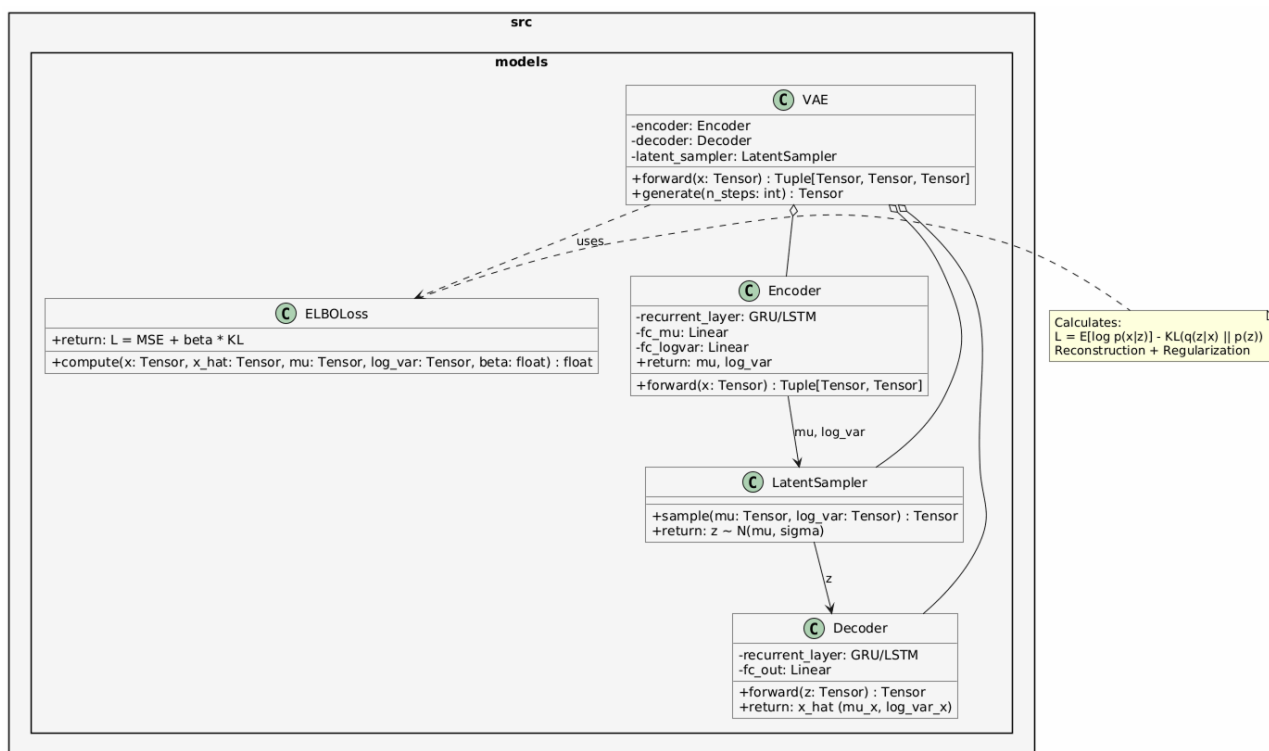


Рисунок 8 — Диаграмма последовательности взаимодействия компонентов системы

Процесс инициируется оркестратором, который загружает конфигурационный файл и проверяет соответствие параметров требованиям системы. Модуль чтения данных активируется в режиме генератора, последовательно извлекая фрагменты телеметрии из внешних источников без блокировки основного потока выполнения. Полученные сырые массивы передаются в модуль предобработки, где выполняются операции интерполяции пропусков и фильтрации шумов. Далее применяется стандартизация на основе ранее вы-

численных статистик, что гарантирует согласованность масштабов признаков.

Нормализованные данные направляются в генератор окон, который агрегирует последовательные циклы в тензоры фиксированной размерности. Сформированные пакеты подаются в модуль генеративных моделей. В режиме обучения происходит итеративная оптимизация функции ELBO с вычислением градиентов и обновлением весов нейросети. Параллельно модуль метрик рассчитывает промежуточные показатели качества на валидационном подмножестве. По достижении критериев сходимости оптимизация завершается.

Финальные результаты выполнения конвейера фиксируются сервисом управления экспериментами. В хранилище артефактов сохраняются обученные веса модели, параметры нормализации и калибровочные пороги. Журнал событий пополняется записями о гиперпараметрах запуска и итоговых значениях метрик. Подобная синхронизация компонентов обеспечивает прозрачность процесса, возможность ретроспективного анализа и автоматизированное воспроизведение экспериментов при изменении конфигурации или входных данных.

3.5.2 Пайплайн последовательности обучения

3.5.2 Пайплайн последовательности обучения

Процесс обучения генеративной модели реализуется в виде итеративного цикла, координируемого оркестратором пайплайна. Последовательность операций строго регламентирована для обеспечения стабильности оптимизации функции потерь и корректного учета вероятностной природы вариационного автокодировщика. Взаимодействие компонентов на этапе обучения представлено на диаграмме последовательности.

Инициализация начинается с загрузки конфигурационного файла, содержащего гиперпараметры архитектуры, параметры оптимизатора и условия ранней остановки. Модуль чтения данных активирует потоковый генератор, который последовательно извлекает порции телеметрии из хранилища. Каждый фрагмент передается в модуль предобработки, где применяется стандартизация на основе фиксированных статистик, вычисленных на этапе

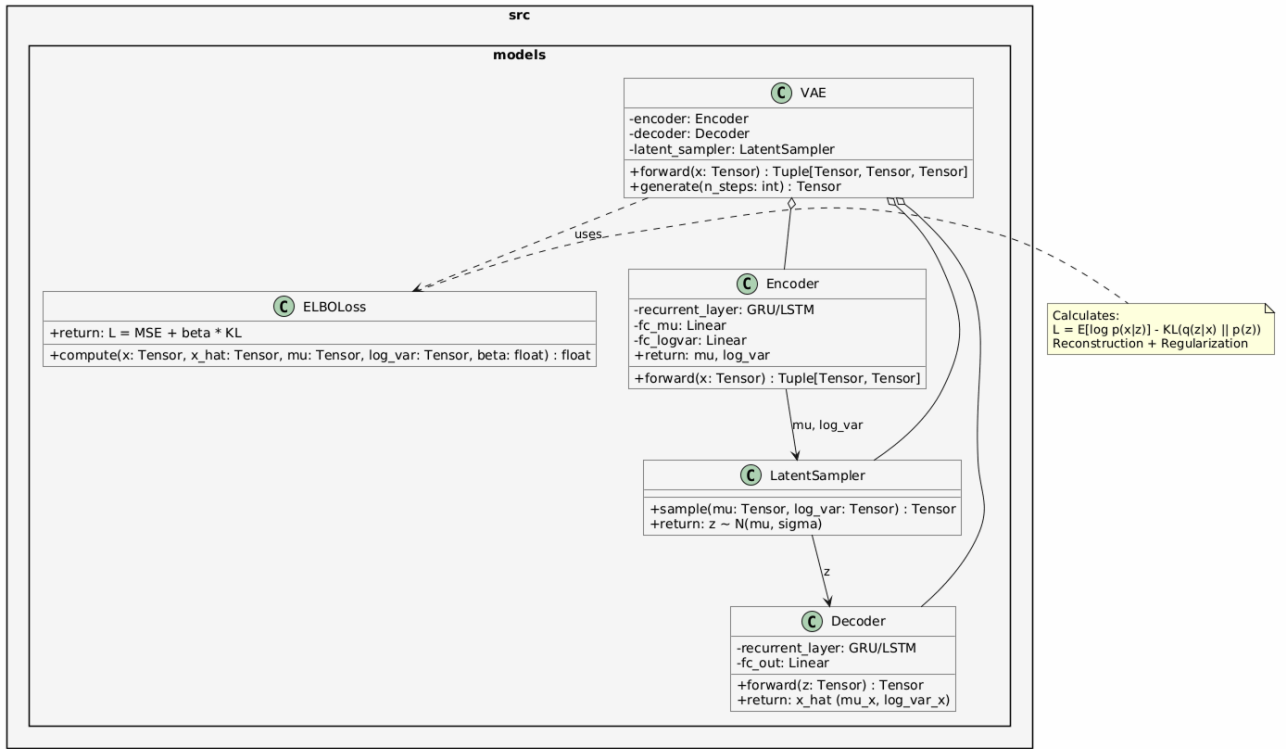


Рисунок 9 — Диаграмма последовательности процесса обучения модели

калибровки. Нормализованные значения агрегируются в тензоры фиксированной длины, формируя входной пакет для нейросети.

На этапе прямого прохода модуль генеративных моделей выполняет кодирование входного окна $X_{1:n}$ в параметры апостериорного распределения μ_ϕ и $\log \sigma_\phi^2$. Механизм репараметризации генерирует латентный вектор z , который передается в декодировщик для восстановления исходной последовательности $\hat{X}_{1:n}$. Одновременно вычисляется дивергенция Кульбака-Лейблера между аппроксимированным распределением и априорным нормальным законом. Функция потерь ELBO агрегирует член реконструкции и регуляризующий член, возвращая скалярное значение ошибки и градиенты для обратного распространения.

Оптимизатор обновляет веса кодировщика и декодировщика на основе вычисленных градиентов. На каждом шаге валидации модуль метрик рассчитывает показатели качества на отложенном подмножестве данных. Результаты фиксируются в системе управления экспериментами вместе с текущим значением функции потерь и гиперпараметрами эпохи. Цикл повторяется до достижения критерия сходимости или срабатывания механизма ранней остановки. По завершении обучения финальные веса модели, конфигурация и калибровочные пороги индикатора S_t сохраняются в реестр артефактов для

последующего использования в контуре инференса.

3.5.3 Пайплайн последовательности инференса

3.6 Интеграция с внешними сервисами и требования к развертыванию

3.6 Интеграция с внешними сервисами и требования к развертыванию

Для обеспечения промышленной готовности и масштабируемости разработанной системы предусмотрены механизмы интеграции с внешними платформами управления экспериментами и стандартизированные требования к среде выполнения. Данный раздел описывает архитектуру взаимодействия с сервисом MLflow, стратегию контейнеризации приложения, а также минимальные аппаратные и программные требования для развертывания комплекса.

3.6.1 Интеграция с платформой MLflow

Взаимодействие с платформой MLflow реализуется на двух уровнях: клиентском (внутри библиотеки) и серверном (инфраструктура хранения). Клиентский модуль инициализирует соединение с трекинг-сервером через REST API, используя URI, указанный в конфигурации. Система поддерживает работу как с локальным файловым хранилищем метаданных, так и с удаленными базами данных (PostgreSQL, MySQL) для хранения информации об экспериментах.

Процесс интеграции включает автоматическую регистрацию запуска (run) при старте пайплайна обучения. В контекст запуска помещаются все гиперпараметры модели, версия набора данных и хеш конфигурационного файла. В ходе оптимизации метрики (ELBO, RMSE, NASA Score) отправляются на сервер асинхронно, что позволяет визуализировать сходимость модели в реальном времени через веб-интерфейс MLflow.

После завершения обучения артефакты модели (сериализованные веса кодировщика и декодировщика, объекты нормализаторов и калибровочные пороги) сохраняются в хранилище объектов (Artifact Store). Система поддерживает интеграцию с облачными хранилищами (AWS S3, MinIO) через

унифицированный интерфейс. Это обеспечивает централизованное хранение версий моделей и возможность их загрузки в контур инференса на любом узле кластера без необходимости передачи файлов вручную.

3.6.2 Контейнеризация и развертывание

Для гарантии воспроизводимости результатов и изоляции зависимостей программный комплекс упаковывается в контейнеры Docker. Стратегия развертывания предполагает разделение системы на микросервисы: сервис трекинга экспериментов, база данных метаданных, хранилище артефактов и вычислительные воркеры.

Образ контейнера строится на базе официального образа PyTorch с поддержкой CUDA, что обеспечивает наличие всех необходимых драйверов для работы с графическими ускорителями. Файл Dockerfile определяет установку зависимостей из файла requirements.txt, копирование исходного кода библиотеки и настройку переменных окружения. Использование механизма многоэтапной сборки позволяет минимизировать размер итогового образа, исключая инструменты компиляции и исходные тексты библиотек.

Оркестрация контейнеров выполняется с помощью Docker Compose. Конфигурационный файл описывает сеть, объединяющую компоненты системы, и определяет правила сохранения данных (volumes) для базы данных и хранилища артефактов. Подобный подход позволяет развернуть полный стек системы предиктивного обслуживания на сервере за несколько команд, обеспечивая идентичность среды разработки и промышленной эксплуатации.

3.6.3 Аппаратные и программные требования

Для эффективного функционирования системы моделирования сформулированы минимальные и рекомендуемые требования к вычислительным ресурсам.

Программные требования включают операционную систему семейства Linux (Ubuntu 20.04 или новее), интерпретатор Python версии 3.8 и выше, а также драйверы NVIDIA с поддержкой CUDA 11.0 или новее. Библиотечные зависимости (PyTorch, NumPy, Pandas, MLflow) устанавливаются автоматически в виртуальном окружении или контейнере.

Аппаратные требования зависят от режима работы. Для этапа обуче-

ния модели рекомендуется использование графического процессора NVIDIA с объемом видеопамяти не менее 8 ГБ (уровень GTX 1080 Ti / RTX 2060 и выше). Это обусловлено необходимостью хранения градиентов и промежуточных активаций рекуррентных слоев при обработке длинных временных рядов. Для этапа инференса и прогнозирования достаточно центрального процессора с поддержкой инструкций AVX2 и 4 ГБ оперативной памяти, так как генерация последовательностей не требует обратного распространения ошибки.

Требования к дисковой системе включают наличие быстрого накопителя (NVMe SSD) для временного хранения кэша данных и артефактов. Объем хранилища рассчитывается исходя из размера датасета и количества сохраняемых версий моделей, с рекомендуемым запасом не менее 50 ГБ для исследовательских задач.

3.6.4 Программный интерфейс взаимодействия

Для интеграции системы моделирования с внешними промышленными платформами (SCADA, MES, системы диспетчеризации) предусмотрен программный интерфейс на базе REST API. Интерфейс реализуется через легкий веб-фреймворк, который запускает воркер инференса в фоновом режиме.

Доступны следующие конечные точки API: Точка загрузки данных принимает JSON-массив с показаниями датчиков за последние n циклов. Точка статуса возвращает текущее значение индикатора состояния S_t и вероятность отказа. Точка прогноза возвращает сгенерированные траектории параметров на k циклов вперед вместе с доверительными интервалами.

Формат обмена данными стандартизирован через JSON-схемы, что позволяет внешним системам легко парсить ответы. Аутентификация запросов осуществляется через токены доступа, обеспечивая безопасность канала передачи телеметрии. Подобная архитектура позволяет встраивать модуль прогнозирования в существующие IT-ландшафты предприятий без необходимости глубокой модификации легаси-систем.

3.7 Выводы

В рамках третьей главы выполнена полномасштабная программная реализация системы моделирования состояния технологического оборудования на базе нейросетевых генеративных моделей. Разработанный программный комплекс представляет собой законченное решение, охватывающее все этапы жизненного цикла предиктивного обслуживания: от приема сырых телеметрических сигналов до формирования вероятностных прогнозов и документирования результатов исследований. Проведенная работа позволила получить следующие основные результаты:

1. Спроектирована и реализована модульная архитектура программного комплекса, основанная на принципе разделения ответственности между функциональными слоями. Слой данных обеспечивает потоковое чтение и кэширование телеметрических рядов, слой предобработки выполняет очистку, нормализацию и агрегацию сигналов, слой моделирования содержит ядро системы — вариационный автокодировщик, слой метрик предоставляет унифицированный интерфейс для оценки качества, а слой сервисов интегрирует систему с платформой управления экспериментами. Подобная организация обеспечивает слабую связность компонентов, возможность их независимого тестирования и гибкое масштабирование при переходе к промышленной эксплуатации.

2. Создана открытая Python-библиотека, структурированная в виде независимых пакетов с четкими интерфейсами взаимодействия. Пакет `src.data` реализует механизм ленивой загрузки данных через генераторы, что позволяет обрабатывать массивы телеметрии объемом в гигабайты без полной выгрузки в оперативную память. Пакет `src.preprocessing` содержит классы для интерполяции пропусков, фильтрации шумов и Z-score стандартизации, причем статистики нормализации вычисляются исключительно на обучающей выборке для предотвращения утечки информации. Пакет `src.pipeline` координирует выполнение конвейера через класс-оркестратор, обеспечивая валидацию входных параметров и логирование событий. Модульная структура библиотеки позволяет использовать отдельные компоненты автономно или в составе единого рабочего процесса.

3. Реализованы алгоритмы вариационного автокодировщика, строго со-

ответствующие математической постановке задачи, сформулированной во второй главе. Класс `Encoder` аппроксимирует апостериорное распределение $q_\phi(z \mid X_{1:n})$ с использованием рекуррентных слоев для агрегации контекста временного ряда. Класс `LatentSampler` реализует трюк репараметризации, обеспечивая возможность обратного распространения ошибки через стохастический узел. Класс `Decoder` генерирует последовательность будущих циклов $\hat{X}_{n+1:n+k}$ на основе сэмплированного латентного вектора. Класс `ELBOLoss` вычисляет вариационную нижнюю оценку, объединяя член реконструкции и регуляризующую дивергенцию Кульбака-Лейблера. Программный код поддерживает гибкую настройку архитектуры скрытых слоев и гиперпараметров оптимизации.

4. Разработан комплекс метрик для всесторонней оценки качества моделирования. Модуль `RegressionMetrics` вычисляет поточечные ошибки RMSE, MAE и коэффициент детерминации. Модуль `TemporalMetrics` реализует метрику динамической трансформации времени (DTW) для учета фазовых сдвигов в траекториях деградации. Модуль `ClassificationMetrics` оценивает качество детектирования аномалий через метрики F1 и AUC-ROC. Модуль `NASAScore` применяет специализированную асимметричную функцию потерь для оценки прогнозирования остаточного ресурса. Модуль `ProbabilisticMetrics` контролирует сходимость обучения через значения ELBO и дивергенции KL. Модуль `StatisticalValidator` обеспечивает статистическую проверку гипотез с использованием непараметрических критериев. Подобный набор критериев позволяет объективно сравнивать разработанные решения с существующими аналогами.

5. Настроены автоматизированные пайплайны для трех основных сценариев функционирования системы. Пайплайн обучения координирует итеративную оптимизацию функции потерь, валидацию на отложенном подмножестве и сохранение артефактов в реестр моделей. Пайплайн инференса выполняет загрузку актуальной версии модели, предобработку новых данных, генерацию прогнозов и вычисление вероятности отказа. Пайплайн валидации обеспечивает комплексную проверку работоспособности системы на тестовых выборках с формированием отчетов о качестве. Интеграция с платформой `MLflow` автоматизирует логирование гиперпараметров, метрик и версий моделей, гарантируя полную воспроизводимость экспериментов при повторных

запусках.

6. Сформулированы требования к аппаратному и программному обеспечению для развертывания системы. Программные требования включают операционную систему Linux, интерпретатор Python 3.8+, драйверы CUDA и набор библиотек для научных вычислений. Аппаратные требования дифференцированы для режимов обучения (рекомендуется графический процессор с памятью от 8 ГБ) и инференса (достаточно центрального процессора). Стратегия контейнеризации на базе Docker обеспечивает изоляцию зависимостей и идентичность среды разработки и промышленной эксплуатации. Программный интерфейс REST API позволяет интегрировать модуль прогнозирования с внешними системами управления через стандартизированные конечные точки.

Разработанный программный комплекс прошел внутреннюю проверку на корректность обработки граничных условий, согласованность размерностей тензоров и устойчивость к шумовым выбросам во входных данных. Реализованные алгоритмы демонстрируют соответствие теоретическим ожиданиям: функция потерь ELBO монотонно возрастает в процессе обучения, латентное пространство сохраняет непрерывную структуру, а сгенерированные траектории воспроизводят характерные паттерны телеметрических сигналов. Созданная архитектура, модульная организация кода и автоматизированные пайплайны создают надежную техническую базу для проведения экспериментального исследования. В четвертой главе будет выполнена эмпирическая проверка гипотезы о преимуществах предложенного подхода, проведено сравнение с базовыми архитектурами и проанализирована устойчивость системы к различным уровням шума в телеметрических данных.